Finite Automata
NFA → DFA

Full lecture notes with exam practice

Mode: English (default)
Content: full notes + 12 / 10 / 6 / 10 questions
Source: original sample notes written for this example

LECTOPRINT

0 Contents

1	Deterministic Finite Automata
2	Nondeterministic Finite Automata
3	Subset Construction
4	ε-NFAs and ε-closure
5	Glossary
6	Exam Practice
7	Key Exam Points

1 Deterministic Finite Automata

1.1 What and why

A finite automaton is the simplest model of a machine with memory: it has a fixed, finite number of states and reads its input one symbol at a time. **Deterministic** means there is never a choice: from each state, each symbol leads to exactly one next state. This makes a DFA easy to run in hardware or code, and it is the reference model the rest of the chapter is compared against.

SOURCE · Sample notes, p.1
A DFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where Q is a finite set of states, Σ is the input alphabet, $\delta : Q \times \Sigma \to Q$ is the transition function, $q_0 \in Q$ is the start state and $F \subseteq Q$ is the set of accepting states.

Q	Finite set of states.
Σ	Input alphabet: a finite, non-empty set of symbols.
δ	Total function Q × Σ → Q; exactly one next state per (state, symbol).
q_0	The single start state.
F	Accepting states; may be empty.

A DFA accepts a string w when the run that starts in q_0 and follows w ends in F. Using the extended transition function:

$$\hat{\delta}(q, \varepsilon) = q, \qquad \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$$

$$w \in L(A) \Leftrightarrow \hat{\delta}(q_0, w) \in F$$

WARNING:
If $F = \varnothing$ the DFA accepts nothing, not even the empty string ε. A DFA with no accepting states recognises the empty language ∅.

2 Nondeterministic Finite Automata

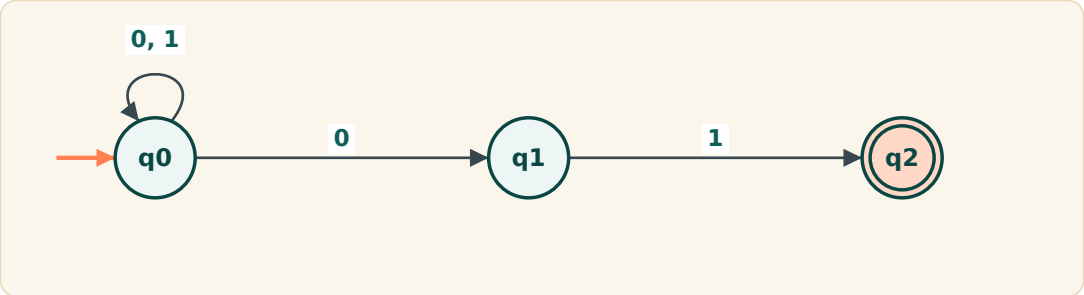
2.1 What and why

Many languages are easier to describe by *guessing*. For "strings that end in 01" it is natural to say: wait, and at some point guess that the final 01 starts here. An NFA allows exactly this: from one state, a symbol may lead to several states, or to none. The NFA accepts if **at least one** of its possible runs ends in an accepting state.

SOURCE · Sample notes, p.2
An NFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ defined like a DFA, except that $\delta : Q \times \Sigma \to P(Q)$. The NFA accepts w if at least one run on w ends in a state of F.

2.2 Example: strings that end in 01

Example 1 (source, p.2)
Build an NFA over Σ = {0, 1} that accepts exactly the strings ending in 01.



NFA N for strings ending in 01 (redrawn, p.2)

q_0 reads any prefix. On a 0 it may also *guess* that this 0 is the second-to-last symbol and move to q_1; a following 1 reaches the accepting state q_2. If the guess was wrong, that run simply dies: q_1 has no move on 0 and q_2 has no moves at all.

TIP:
When designing an NFA for "ends in …", keep a self-loop on the start state and spell out the suffix as a chain of states. The loop absorbs the prefix; nondeterminism picks where the suffix starts.

3 Subset Construction

3.1 What and why

NFAs are convenient, but a machine cannot run on guesses. The subset construction removes the guessing: the DFA keeps track of **every state the NFA could currently be in**. A DFA state is therefore a set of NFA states, and its next state is the set of all places those states can move to.

SOURCE · Sample notes, p.3
Given an NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$, the subset construction builds a DFA D whose states are subsets of Q_N, with start state $\{q_0\}$, transitions $\delta_D(S, a) = \cup_{p \in S} \delta_N(p, a)$, and accepting states all S with $S \cap F_N \neq \emptyset$.

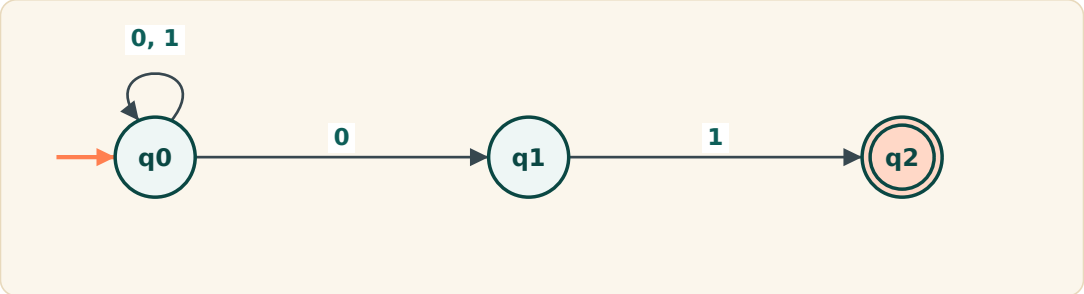
$$\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$$

$$F_D = \{\, S \subseteq Q_N : S \cap F_N \neq \emptyset \,\}$$

$$|Q_D| \leq 2^{|Q_N|}$$

3.2 Example: converting N

Example 2 (source, p.3)
Convert the NFA N of Example 1 to a DFA using the subset construction. Build only the reachable states.



NFA N again, for reference (redrawn, p.2)

Step 1: start state

```
start = {q0}
```

Step 2: moves from {q0}

```
δD({q0}, 0) = δN(q0, 0) = {q0, q1} new
δD({q0}, 1) = δN(q0, 1) = {q0}
```

Step 3: moves from {q0, q1}

```
δD({q0,q1}, 0) = {q0, q1} ∪ ∅ = {q0, q1}
δD({q0,q1}, 1) = {q0} ∪ {q2} = {q0, q2} new
```

Step 4: moves from {q0, q2}

```
δD({q0,q2}, 0) = {q0, q1} ∪ ∅ = {q0, q1}
δD({q0,q2}, 1) = {q0} ∪ ∅ = {q0}
```

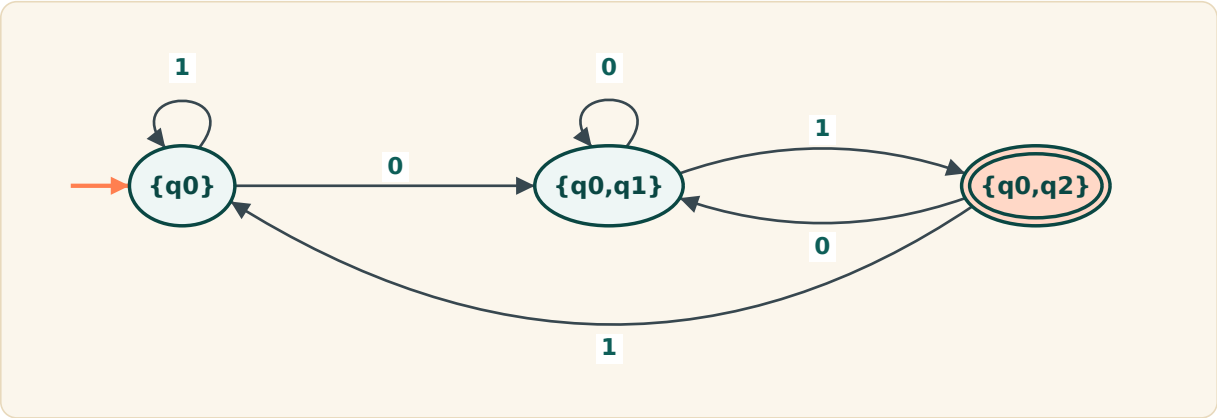
No new sets appeared in Step 4, so the construction stops with three states.

Step 5: accepting states and table

Only $\{q_0, q_2\}$ contains the NFA accepting state q_2, so it is the only accepting DFA state. → marks the start state, * an accepting state.

DFA state	0	1
→ {q0}	{q0, q1}	{q0}
{q0, q1}	{q0, q1}	{q0, q2}
* {q0, q2}	{q0, q1}	{q0}

Transition table of D (green: accepting)



DFA D produced by the subset construction

Check
A script ran D and the original definition ("ends in 01") on every binary string up to length 8; they agree on all of them.

WARNING:
$2^3 = 8$ is only the upper bound. Sets such as $\{q_1\}$ or $\varnothing$ are never reached here, so they are not states of D. Listing them is a common exam mistake.

4 ε-NFAs and ε-closure

4.1 What and why

Some constructions are simpler if the automaton may also move **without reading a symbol**. These ε-moves add no power, but the subset construction must account for them by closing every set under ε-moves.

SOURCE · Sample notes, p.4
ε-closure(q) is the set of states reachable from q by zero or more ε-transitions. For an ε-NFA the subset construction starts with ε-closure(q_0) and uses $\delta_D(S, a) = \varepsilon\text{-closure}(\cup_{p \in S} \delta_N(p, a))$.

TIP:
ε-closure(q) always contains q, because zero moves are allowed.

5 Glossary

Definitions follow the source. **Bold** words are the keywords an exam answer must contain.

How to study this section
(1) Read the term and say the definition in your own words. (2) Compare with the table and find the bold word you missed. (3) Ask why that word is there: for a DFA, why must δ be **total**? Because a DFA has to know its next state for every symbol.

Term	Definition	Source
DFA	A 5-tuple $(Q, \Sigma, \delta, q_0, F)$ with a **total** transition function $\delta : Q \times \Sigma \to Q$.	p.1
NFA	Like a DFA, but $\delta : Q \times \Sigma \to$ **P(Q)**; accepts if **some** run ends in F.	p.2
Subset construction	Builds a DFA whose states are **sets of NFA states**.	p.3
Dead state	A **non-accepting** state that only loops to itself; ∅ in the construction.	p.3
ε-closure	States reachable using **only ε-moves**, including the state itself.	p.4

Commonly confused pairs

Pair	Distinguishing point
DFA vs *NFA*	*exactly one* next state vs *a set* of next states
all runs vs *some run*	NFA acceptance needs *some run* in F, not all
2^n vs *reachable*	2^n is an *upper bound*; the construction builds only *reachable* sets

6 Exam Practice

Note
The source has no question bank of its own, so every question here is an **Original question** built from its definitions and examples. Each answer cites the page it rests on.

6.1 True / False (12)

T/F 1
In a DFA, every state has exactly one transition for each input symbol.

True. δ is a total function $Q \times \Sigma \to Q$. (p.1)

T/F 2
An NFA accepts a string only if all of its computation paths end in an accepting state.

False. One accepting path is enough. (p.2)

T/F 3
For an NFA with n states, the subset construction gives a DFA with at most 2^n states.

True. Each DFA state is a subset of Q_N, and there are 2^n subsets. (p.3)

T/F 4
NFAs recognise a strictly larger class of languages than DFAs.

False. Subset construction turns every NFA into an equivalent DFA; both recognise exactly the regular languages. (p.3)

T/F 5
A DFA state built by the subset construction is accepting if it contains at least one accepting NFA state.

True. $F_D = \{ S \subseteq Q_N : S \cap F_N \neq \emptyset \}$. (p.3)

T/F 6
For an NFA without ε-moves, the start state of the constructed DFA is $\{q_0\}$.

True. The DFA starts in the set that contains only the NFA's start state. (p.3)

T/F 7
The empty set can never appear as a DFA state in the subset construction.

False. $\emptyset$ appears whenever no NFA state in S has a move on a symbol; it becomes the dead state. (p.3)

T/F 8
A DFA may have more than one start state.

False. The 5-tuple has a single start state $q_0 \in Q$. (p.1)

T/F 9
The transition function of an NFA returns a set of states.

True. $\delta_N : Q \times \Sigma \to P(Q)$. (p.2)

T/F 10
A DFA whose set of accepting states F is empty accepts the empty string.

False. With $F = \emptyset$ no string, including ε, is accepted; $L(A) = \emptyset$. (p.1)

T/F 11
Every DFA can be seen as an NFA.

True. Replace $\delta(q, a) = p$ by $\delta_N(q, a) = \{p\}$. (p.2)

T/F 12
The subset construction always produces exactly 2^n states.

False. 2^n is only an upper bound; building only reachable subsets usually gives far fewer. (p.3)

6.2 Multiple Choice (10)

MC 1
Which of the following is NOT part of the 5-tuple of a DFA?
a) a stack alphabet Γ
b) the input alphabet Σ
c) the set of states Q
d) the set of accepting states F

Answer: a. A DFA is $(Q, \Sigma, \delta, q_0, F)$; a stack belongs to pushdown automata. (p.1)

MC 2
In the 'ends in 01' example, what is $\delta_D(\{q_0\}, 0)$?
a) {q0}
b) {q1}
c) {q0, q1}
d) {q0, q2}

Answer: c. $\delta_N(q_0, 0) = \{q_0, q_1\}$. (p.3)

MC 3
Which DFA state is accepting in the 'ends in 01' example?
a) {q0}
b) {q0, q1}
c) {q0, q2}
d) none of them

Answer: c. Only $\{q_0, q_2\}$ contains the NFA accepting state q_2. (p.3)

MC 4
At most how many states can the subset construction produce from a 4-state NFA?
a) 16
b) 4
c) 8
d) 32

Answer: a. $2^4 = 16$ subsets. (p.3)

MC 5
An NFA accepts a string w if...
a) every path for w ends in F
b) some path for w ends in F
c) the path for w visits every state
d) no path for w gets stuck

Answer: b. Existence of one accepting path is the acceptance condition. (p.2)

MC 6
What is the type of an NFA's transition function?
a) $Q \times \Sigma \to Q$
b) $Q \to \Sigma$
c) $P(Q) \times \Sigma \to Q$
d) $Q \times \Sigma \to P(Q)$

Answer: d. Each (state, symbol) pair maps to a set of states. (p.2)

MC 7
In the subset construction, $\delta_D(S, a)$ is computed as…
a) the intersection of $\delta_N(p, a)$ for $p \in S$
b) the union of $\delta_N(p, a)$ for $p \in S$
c) δ_N of the first state in S
d) the complement of S

Answer: b. $\delta_D(S, a) = \cup_{p \in S} \delta_N(p, a)$. (p.3)

MC 8
Which string is accepted by the 'ends in 01' automaton?
a) 10
b) 0110
c) 110
d) 1001

Answer: d. Only 1001 ends in 01. (p.3)

MC 9
In the subset construction, the dead (trap) state corresponds to…
a) {q0}
b) ∅
c) Q
d) F

Answer: b. The empty set has no outgoing moves except back to itself and is never accepting. (p.3)

MC 10
Which statement about the power of DFAs and NFAs is correct?
a) NFAs are more powerful
b) DFAs are more powerful
c) they are incomparable
d) they recognise the same languages

Answer: d. Every NFA has an equivalent DFA and vice versa. (p.3)

6.3 Open-ended (6)

OE 1
Explain why each state of the DFA built by the subset construction is a set of NFA states.

Model answer
After reading a prefix, an NFA can be in **several states at once**. A DFA must be in exactly one state, so the DFA state records **the whole set** of NFA states the NFA could be in.
Because the set fully describes the NFA's situation, the DFA can compute the next set with $\delta_D(S, a) = \cup_{p \in S} \delta_N(p, a)$ (p.3).

OE 2 (Scenario)
A digital lock reads a stream of bits and opens when the last two bits are 01. Design an NFA for it, convert it to a DFA and give the transition table.

Model answer
NFA: q_0 loops on 0 and 1 and guesses the start of the suffix with q_0 –0→ q_1; then q_1 –1→ q_2; F = $\{q_2\}$.
DFA table: {q0}: 0→{q0,q1}, 1→{q0}; {q0,q1}: 0→{q0,q1}, 1→{q0,q2}; {q0,q2}: 0→{q0,q1}, 1→{q0}. Start {q0}; accepting {q0,q2} because it contains q_2 (p.3).

OE 3
Explain the role of the empty set ∅ in the subset construction and when it appears.

Model answer
∅ appears when **no state in S** has a move on symbol a. It is a **dead state**: every symbol leads back to ∅ and it is never accepting.
It keeps δ_D **total**, which a DFA requires (p.1, p.3).

OE 4
Compare DFAs and NFAs in terms of transition function, acceptance condition and expressive power.

Model answer
Transition function: DFA $\delta : Q \times \Sigma \to Q$ (one next state); NFA $\delta : Q \times \Sigma \to P(Q)$ (a set, possibly empty).
Acceptance: DFA accepts if **the** run ends in F; NFA accepts if **some** run ends in F.
Expressive power: **equal**; both recognise exactly the regular languages, by the subset construction (p.1–3).

OE 5 (Scenario)
A text filter must flag every string over {a, b} that contains ab as a substring. Design an NFA, convert it to a DFA and name the accepting DFA states.

Model answer
NFA: p_0 loops on a, b; p_0 –a→ p_1; p_1 –b→ p_2; p_2 loops on a, b; F = {p_2}.
DFA: {p0}: a→{p0,p1}, b→{p0}; {p0,p1}: a→{p0,p1}, b→{p0,p2}; {p0,p2}: a→{p0,p1,p2}, b→{p0,p2}; {p0,p1,p2}: a→{p0,p1,p2}, b→{p0,p2}.
Accepting: {p0,p2} and {p0,p1,p2}, the sets that contain p_2 (p.3).

OE 6
Explain why the language 'the third symbol from the end is 1' over {0, 1} needs 8 DFA states but only 4 NFA states.

Model answer
The NFA **guesses** the position of that 1 and counts two more symbols: 4 states.
A DFA cannot guess, so it must **remember the last three symbols**: 2^3 = 8 combinations, and any two different combinations must lead to different states.
This is the 2^n blow-up of the subset construction in a case where it cannot be avoided (p.3).

6.4 Definition Questions (10)

How to answer
First write the definition with its keywords, then add an example or drawing rule from the source.

Definition 1
Define a deterministic finite automaton (DFA).

Model answer
A DFA is a 5-tuple (Q, Σ, δ, q_0, F): a **finite set of states** Q, an **input alphabet** Σ, a **total transition function** δ : Q × Σ → Q, a **start state** $q_0 \in Q$ and a set of **accepting states** $F \subseteq Q$ (p.1).

Definition 2
Define a nondeterministic finite automaton (NFA).

Model answer
An NFA is a 5-tuple (Q, Σ, δ, q_0, F) like a DFA, except that δ : Q × Σ → **P(Q)** returns a **set of states**. It accepts w if **at least one** run on w ends in F (p.2).

Definition 3
Distinguish the transition function of a DFA from that of an NFA.

Model answer
DFA: δ(q, a) is **exactly one** state. NFA: δ(q, a) is a **set** of states that may be **empty** or contain several states (p.1–2).

Definition 4
Define the subset construction.

Model answer
The subset construction builds a DFA from an NFA whose states are **sets of NFA states**: start $\{q_0\}$, $\delta_D(S, a) = \mathbf{\cup_{p \in S}\ \delta_N(p, a)}$, and S is accepting if $\mathbf{S \cap F_N \neq \emptyset}$ (p.3).

Definition 5
Define ε-closure.

Model answer
ε-closure(q) is the set of states reachable from q using **only ε-moves**, **including q itself** (zero moves). For a set, take the union over its members (p.4).

Definition 6
Define the language of an automaton L(A).

Model answer
L(A) is the **set of all strings** over Σ that A **accepts** (p.1).

Definition 7
Define accepting (final) states.

Model answer
The accepting states F ⊆ Q are the states in which a run must **end** for the input to be **accepted**; F may be empty (p.1).

Definition 8
Define a dead (trap) state.

Model answer
A dead state is a **non-accepting** state whose every transition **leads back to itself**; once entered, the input can no longer be accepted. In the subset construction it is ∅ (p.3).

Definition 9
Define the extended transition function δ̂.

Model answer
δ̂ extends δ from **symbols to strings**: δ̂(q, ε) = q and δ̂(q, wa) = δ(δ̂(q, w), a). A DFA accepts w if $\hat{\delta}(q_0, w) \in F$ (p.1).

Definition 10
Define equivalence of two automata.

Model answer
Two automata are equivalent if they **accept the same language**, L(A) = L(B), even if their numbers of states differ (p.3).

7 Key Exam Points

A DFA has **one** next state per symbol; an NFA has a **set**.

NFA accepts if **some** run ends in F.

Subset construction: start $\{q_0\}$, $\delta_D(S, a) = \cup\, \delta_N(p, a)$, accept if $S \cap F \neq \emptyset$.

At most 2^n DFA states; build only the reachable ones.

∅ is the dead state that keeps δ_D total.